

Week5 : Tree Based Methods

Objectives

This homework sheet will help reviewing the basic concepts associated with tree-based methods and provide a basic introduction to the ML workflow using real-world datasets. Please review the lectures, suggested readings, and additional resources *before* getting started on the HW.

Some questions in this assignment will require you to conduct independent research beyond the material covered in the recorded content.

The following website has access to the relevant datasets from the recommended textbook: <https://book.huihoo.com/introduction-to-statistical-learning/data.html>

Marks Distribution

Question	Marks
1	3
2	3
3a	2
3b	2
3c	5

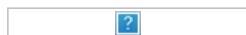
Questions

Conceptual

Q1. Draw an example (of your own invention) of a partition of two-dimensional feature space that could result from recursive binary splitting. Your example should contain at least six regions. Draw a decision tree corresponding to this partition. Be sure to label all aspects of your figures, including the regions $R_1, R^2, \dots$, the cutpoints $t_1, t_2, \dots$, and so forth. Please insert your sketch below.

Please note: You can use Microsoft word / any editor or draw the image by hand, you can upload the image / screenshot to colab file using insert image option in text cell

ANSWER



Explanation:

- The left diagram shows how the feature space is divided based on recursive splitting.
- The right diagram shows the corresponding decision tree structure.
- This visualization effectively explains how decision trees use binary recursive splitting to create regions for classification or regression.

1. Feature Space Splitting (Left Side)

- This is a 2D representation of the feature space divided into distinct regions $(R_1, R_2, R_3, R_4, R_5, R_6)$
- The dataset consists of two features:
 - $X_1 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$
 - $X_2 = [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]$
 - The space is split along different threshold values $(t_1, t_2, t_3, t_4, t_5)$ shown in the different colors.
 - Each split divides the space into 20 subregions, forming a hierarchical partitioning.

2. Decision Tree (Right Side)

- This represents the hierarchical decision-making process that corresponds to the feature space splitting.
- The root node checks $X_1 \leq 5$ (split t_1)
- If true, the decision tree checks $X_2 \leq 6$ (split t_2) leading to R_1 and further subregions.
- If false, another condition $X_2 \leq 12$ (split t_4) is checked.
- The tree continues recursively until reaching terminal regions $(R_1, R_2, R_3, R_4, R_5, R_6)$

Q2. Suppose we produce ten bootstrapped samples from a data set containing red and green classes. We then apply a classification tree to each bootstrapped sample and, for a specific value of X , produce 10 estimates of $P(ClassisRed|X)$:

0.1, 0.15, 0.2, 0.2, 0.55, 0.6, 0.6, 0.65, 0.7, and 0.75.

There are two common ways to combine these results together into a single class prediction. One is the majority vote approach, the second approach is to classify based on the average probability. In this example, what is the final classification under each of these two approaches?

ANSWER

We are Provided with the following information:

- A dataset with two possible classes: Red and Green.
- 10 bootstrapped samples from the dataset.
- A classification tree applied to each bootstrapped sample.
- For a specific value of X , we get 10 probability estimates for $P(\text{Red}|X)$
 - 0.1, 0.15, 0.2, 0.2, 0.55, 0.6, 0.6, 0.65, 0.7, 0.75
 - If $P(\text{Red}|X) > 0.5$ we predict it is Red, otherwise, we predict Green.

Majority Vote Approach:

- In this approach, we classify based on the majority vote:
 - Votes for Green: $P(\text{Red}|X) \leq 0.5 = 4\text{votes}$
 - Votes for Red: $P(\text{Red}|X) > 0.5 = 6\text{votes}$
- Since, the majority vote (6) predict Red, the final classification is **RED**.

Average Probability:

- In this type we take the average of all the probabilities as per below:
 - $$\frac{(0.1+0.15+0.2+0.2+0.55+0.6+0.6+0.65+0.7+0.75)}{10}$$
 - $$= \frac{4.5}{10} = 0.45^{**}$$
- Since, the average probability (0.45) is less than 0.5, we classify as **GREEN**.

Final Answer

Approach	Final Classification
Majority Vote	Red
Average Probability	Green

APPLIED

Q3. Apply boosting and random forest to a data set of your choice. Feel free to use any of the datasets from ISLR dataset link provided (<https://book.huihoo.com/introduction-to-statistical-learning/data.html>) to examine any of the questions that were discussed in any of the previous homeworks. Be sure to fit the models on a training set and to evaluate their performance on a test set. How accurate are the results compared to simple methods (e.g. linear or logistic regression models)? Which of these approaches yields the best performance?

ANSWER

```
In [226...] #!pip install scikit-learn
```

```
In [227...] # import required libraries
import os

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier, GradientBoostingRegressor, GradientBoostingClassifier

from sklearn.metrics import r2_score, mean_squared_error, confusion_matrix, accuracy_score, f1_score, classification_report
```

```
In [228.. # Suppress unnecessary warning messages
import warnings
warnings.filterwarnings('ignore')
```

a) Read the dataset using function read_csv() from pandas library and preprocess the data if required (check for any missing values etc)

```
In [229.. # Load the dataset

df = pd.read_csv('Auto.csv')

# Make a copy and do processing on the copy
df_auto = df.copy()
df_auto.head()
```

```
Out[229..
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	year	origin	name
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino

```
In [230.. df_auto.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 397 entries, 0 to 396
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   mpg             397 non-null   float64
1   cylinders       397 non-null   int64
2   displacement    397 non-null   float64
3   horsepower      397 non-null   object
4   weight          397 non-null   int64
5   acceleration    397 non-null   float64
6   year           397 non-null   int64
7   origin          397 non-null   int64
8   name            397 non-null   object
dtypes: float64(3), int64(4), object(2)
memory usage: 28.0+ KB
```

```
In [231.. print(df_auto.isnull().sum())

mpg             0
cylinders       0
displacement    0
horsepower      0
weight          0
acceleration    0
year            0
origin          0
name            0
dtype: int64
```

```
In [232.. # There are some values containing question marks in horsepower, we need to fix them.

# Identify rows where horsepower == '?'
missing_hp_rows = df_auto[df_auto['horsepower'] == '?']

if len(missing_hp_rows) > 0:
    print(f"\nRows with missing horsepower:")
    print(missing_hp_rows)

# Replace '?' with NaN
df_auto.loc[df_auto['horsepower'] == '?', 'horsepower'] = np.nan

# Convert horsepower column to float
df_auto['horsepower'] = df_auto['horsepower'].astype('float')

# Fill missing horsepower with the mean HP for that car name
df_auto['horsepower'] = df_auto.groupby('name')['horsepower'].transform(
    lambda x: x.fillna(x.mean())
)

# For the remaining values We might want to fill those remaining NaNs with a global mean.
df_auto['horsepower'] = df_auto['horsepower'].fillna(
    df_auto['horsepower'].mean()
)
```

```
print('? values in horsepower is treated')

# Convert horsepower column to int
df_auto['horsepower'] = df_auto['horsepower'].astype('int64')
print('horsepower column is converted to int64')
```

Rows with missing horsepower:

	mpg	cylinders	displacement	horsepower	weight	acceleration	year	\
32	25.0	4	98.0	?	2046	19.0	71	
126	21.0	6	200.0	?	2875	17.0	74	
330	40.9	4	85.0	?	1835	17.3	80	
336	23.6	4	140.0	?	2905	14.3	80	
354	34.5	4	100.0	?	2320	15.8	81	

	origin	name
32	1	ford pinto
126	1	ford maverick
330	2	renault lecar deluxe
336	1	ford mustang cobra
354	2	renault 18i

? values in horsepower is treated
horsepower column is converted to int64

```
In [233.. # check if name is df_auto columns then drop it
if 'name' in df_auto.columns:
    df_auto.drop('name', axis=1, inplace=True)
    print('name column is dropped')
```

name column is dropped

```
In [234.. print(df_auto.info())
print("")
print(df_auto.describe(include='all').T)
```

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 397 entries, 0 to 396

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	mpg	397 non-null	float64
1	cylinders	397 non-null	int64
2	displacement	397 non-null	float64
3	horsepower	397 non-null	int64
4	weight	397 non-null	int64
5	acceleration	397 non-null	float64
6	year	397 non-null	int64
7	origin	397 non-null	int64

dtypes: float64(3), int64(5)

memory usage: 24.9 KB

None

	count	mean	std	min	25%	50%	75%	\
mpg	397.0	23.515869	7.825804	9.0	17.5	23.0	29.0	
cylinders	397.0	5.458438	1.701577	3.0	4.0	4.0	8.0	
displacement	397.0	193.532746	104.379583	68.0	104.0	146.0	262.0	
horsepower	397.0	104.352645	38.280617	46.0	76.0	94.0	125.0	
weight	397.0	2970.261965	847.904119	1613.0	2223.0	2800.0	3609.0	
acceleration	397.0	15.555668	2.749995	8.0	13.8	15.5	17.1	
year	397.0	75.994962	3.690005	70.0	73.0	76.0	79.0	
origin	397.0	1.574307	0.802549	1.0	1.0	1.0	2.0	

	max
mpg	46.6
cylinders	8.0
displacement	455.0
horsepower	230.0
weight	5140.0
acceleration	24.8
year	82.0
origin	3.0

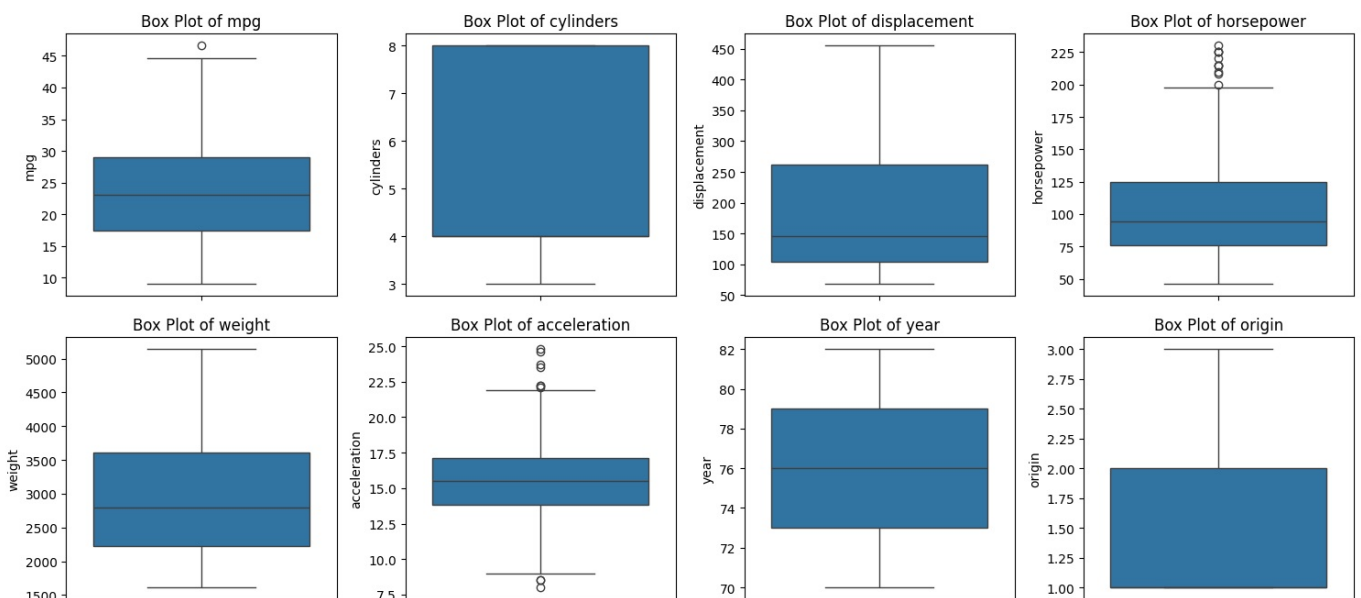
```
In [235.. # Add a new column mpg01 to dataframe
# For the Classifier Models optionally did later
if not 'mpg_class' in df_auto.columns:
    df_auto['mpg_class'] = df_auto['mpg'].apply(
        lambda x: 1 if x > df_auto['mpg'].median() else 0
    )

print(df_auto['mpg_class'].value_counts(), "\n")
df_auto.info()
```

```
mpg_class
0      206
1      191
Name: count, dtype: int64
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 397 entries, 0 to 396
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0    mpg         397 non-null    float64
1    cylinders    397 non-null    int64
2    displacement 397 non-null    float64
3    horsepower   397 non-null    int64
4    weight       397 non-null    int64
5    acceleration  397 non-null    float64
6    year         397 non-null    int64
7    origin       397 non-null    int64
8    mpg_class    397 non-null    int64
dtypes: float64(3), int64(6)
memory usage: 28.0 KB
```

```
In [236.. plt.figure(figsize=(15, 10))
for i, col in enumerate(df_auto.drop(columns=['mpg_class']), 1):
    plt.subplot(3, 4, i) # Adjust rows/columns based on number of features
    sns.boxplot(y=df_auto[col])
    plt.title(f'Box Plot of {col}')
    plt.tight_layout()
plt.show()
```



b) Split the data into train data and test data (proportion of train and test data of your choice)

Reference :

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

We will implement 2 Different Solutions:

We will implement 2 different solutions from here.

1. Required: For Regression Model, which will Predict Sales
2. Optionally: For Classification Model, which will Classify by Sales Class (Low, High)

Train Test Split for Regression

```
In [237.. # For Regression Modles
X_reg = df_auto.drop(columns=['mpg', 'mpg_class'])
y_reg = df_auto['mpg']

X_reg.head(2), y_reg.head(2)
```

```
Out[237...] (   cylinders  displacement  horsepower  weight  acceleration  year  origin
0           8           307.0          130    3504           12.0    70         1
1           8           350.0          165    3693           11.5    70         1,
0          18.0
1          15.0
Name: mpg, dtype: float64)
```

```
In [238...] # For Regression Modles
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)

X_train_reg.shape, y_train_reg.shape, X_test_reg.shape, y_test_reg.shape
```

```
Out[238...] ((317, 7), (317,), (80, 7), (80,))
```

```
In [239...] # For Regression Modles
# Scale the features
scaler = StandardScaler()

X_train_reg_scaled = pd.DataFrame(scaler.fit_transform(X_train_reg), columns=X_reg.columns)
X_test_reg_scaled = pd.DataFrame(scaler.transform(X_test_reg), columns=X_reg.columns)
```

Train Test Split for Classification

```
In [240...] # For Classification Modles
X_class = df_auto.drop(columns=['mpg', 'mpg_class'])
y_class = df_auto['mpg_class']

X_class.head(2), y_class.head(2)
```

```
Out[240...] (   cylinders  displacement  horsepower  weight  acceleration  year  origin
0           8           307.0          130    3504           12.0    70         1
1           8           350.0          165    3693           11.5    70         1,
0          0
1          0
Name: mpg_class, dtype: int64)
```

```
In [241...] # For Classification Models
# Set stratify=y to balance the classes
X_train_class, X_test_class, y_train_class, y_test_class = train_test_split(
    X_class, y_class, test_size=0.3, random_state=10, stratify=y_class
)

X_train_class.shape, y_train_class.shape, X_test_class.shape, y_test_class.shape
```

```
Out[241...] ((277, 7), (277,), (120, 7), (120,))
```

```
In [242...] # For Classification Modles
# Scale the features
scaler = StandardScaler()

X_train_class_scaled = pd.DataFrame(scaler.fit_transform(X_train_class), columns=X_class.columns)
X_test_class_scaled = pd.DataFrame(scaler.transform(X_test_class), columns=X_class.columns)
```

```
In [243...] # For Classification Modles
# Print the classes in training Y and Test Y To check if our split is balance
print("Training Y classes: ", np.unique(y_train_class, return_counts=True))
print("Test Y classes: ", np.unique(y_test_class, return_counts=True))
```

```
Training Y classes: (array([0, 1]), array([144, 133]))
Test Y classes: (array([0, 1]), array([62, 58]))
```

c) Train the data on the below models

i) Linear regression model ii) Gradient boosting model iii) Random forest model

check the train and test r2 score and mention your findings on which model performed better in terms of underfitting / overfitting etc

References:

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

https://scikit-learn.org/stable/auto_examples/ensemble/plot_gradient_boosting_regression.html#gradient-boosting-regression

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.r2_score.html

Regressor Models Implementation:

Predicting Sales using following regressors

```
In [244.. reg_models = {
    "LinearRegression": LinearRegression(),
    "RandomForestRegressor": RandomForestRegressor(random_state=10),
    "GradientBoostingRegressor": GradientBoostingRegressor(random_state=10),
}

In [245.. # Dictionary to store Train & Test performance
reg_results = {}

for name, model in reg_models.items():
    if isinstance(model, LinearRegression):
        model.fit(X_train_reg_scaled, y_train_reg) # Train on scaled data for Linear Regression
        y_pred_reg_train = model.predict(X_train_reg_scaled)
        y_pred_reg_test = model.predict(X_test_reg_scaled)
    else:
        model.fit(X_train_reg, y_train_reg) # Train on raw (unscaled) data for tree-based models
        y_pred_reg_train = model.predict(X_train_reg)
        y_pred_reg_test = model.predict(X_test_reg)

    # Compute R^2 and MSE scores
    r2_train_reg = r2_score(y_train_reg, y_pred_reg_train)
    r2_test_reg = r2_score(y_test_reg, y_pred_reg_test)
    mse_train_reg = mean_squared_error(y_train_reg, y_pred_reg_train)
    mse_test_reg = mean_squared_error(y_test_reg, y_pred_reg_test)

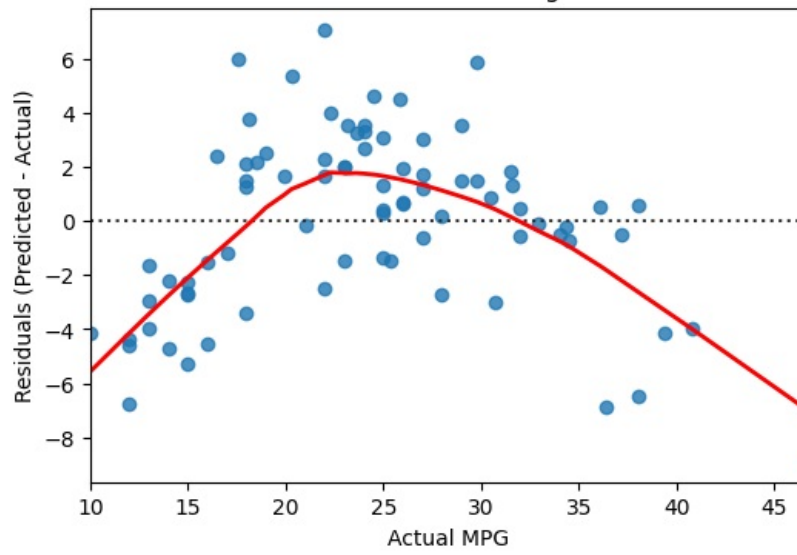
    # Store results
    reg_results[name] = {
        "Train R^2": r2_train_reg, "Test R^2": r2_test_reg,
        "Train MSE": mse_train_reg, "Test MSE": mse_test_reg
    }

    # Residual Plot
    plt.figure(figsize=(6, 4))
    sns.residplot(x=y_test_reg, y=y_pred_reg_test, lowess=True, line_kws={"color": "red", "lw": 2})
    plt.xlabel("Actual MPG")
    plt.ylabel("Residuals (Predicted - Actual)")
    plt.title(f"Residual Plot - {name}")
    plt.show()

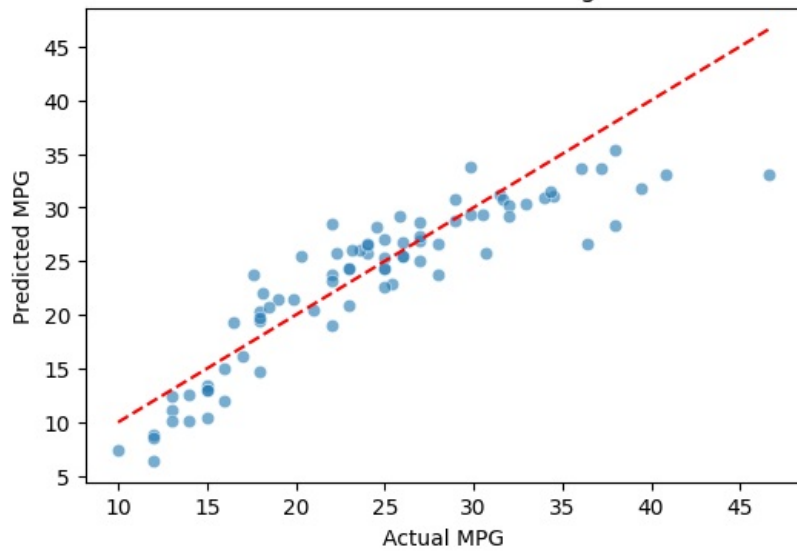
    # Actual vs Predicted Plot
    plt.figure(figsize=(6, 4))
    sns.scatterplot(x=y_test_reg, y=y_pred_reg_test, alpha=0.6)
    plt.plot([y_test_reg.min(), y_test_reg.max()], [y_test_reg.min(), y_test_reg.max()],
             color="red", linestyle="--")
    plt.xlabel("Actual MPG")
    plt.ylabel("Predicted MPG")
    plt.title(f"Actual vs Predicted - {name}")
    plt.show()

    # Feature Importance (For Tree-Based Models)
    if name in ["RandomForestRegressor", "GradientBoostingRegressor"]:
        feature_importances = pd.Series(model.feature_importances_, index=X_reg.columns)
        feature_importances.sort_values(ascending=True).plot(
            kind="barh", figsize=(8, 5), title=f"Feature Importance - {name}")
        plt.show()
```

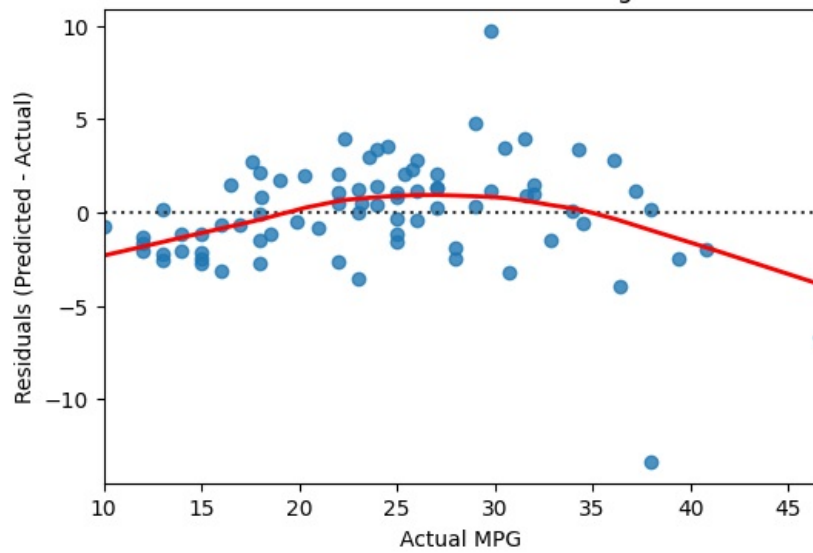
Residual Plot - LinearRegression

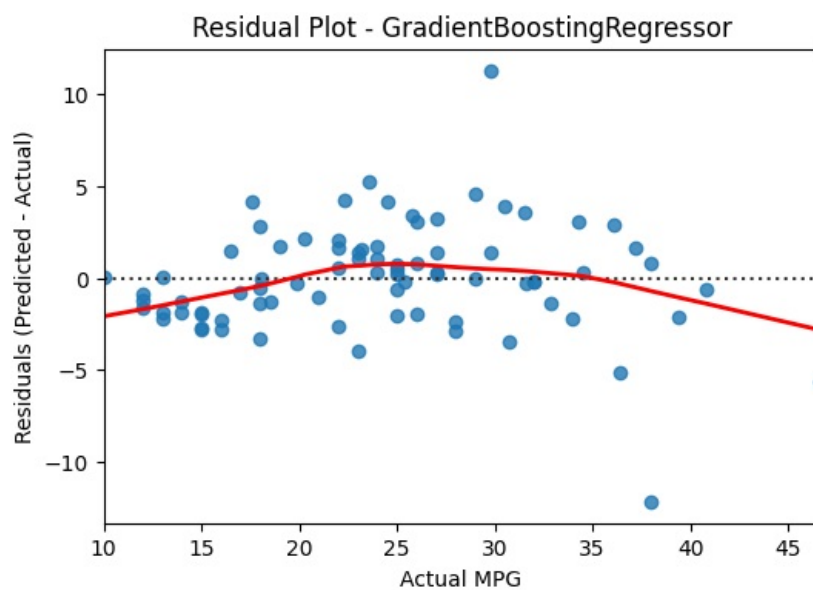
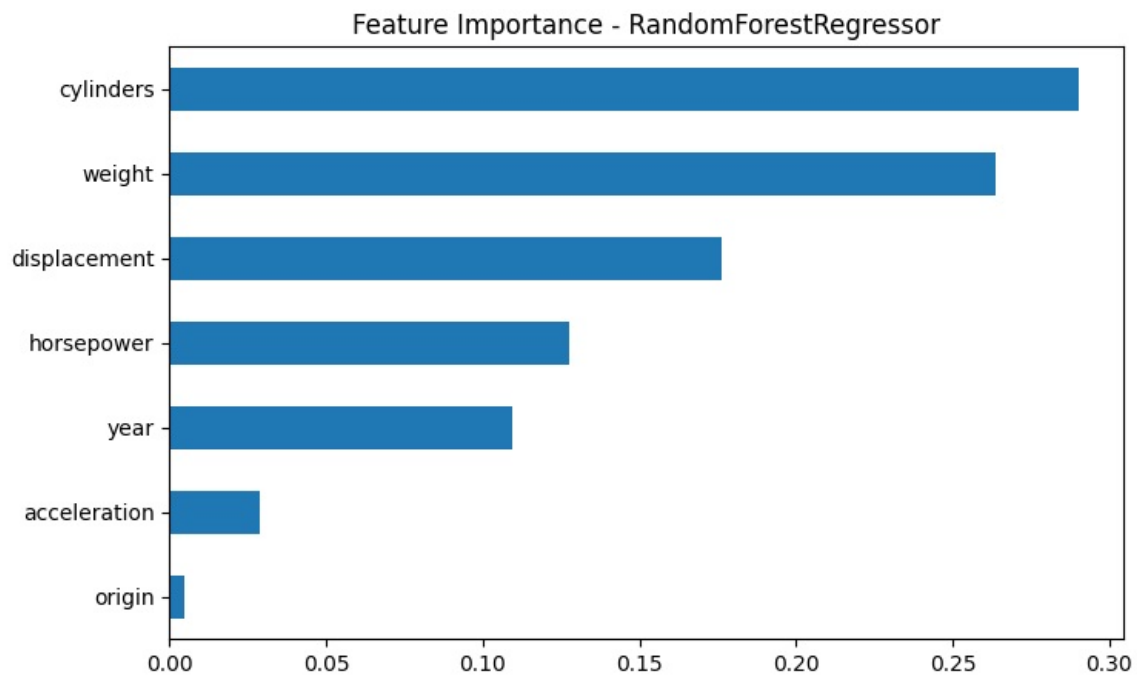
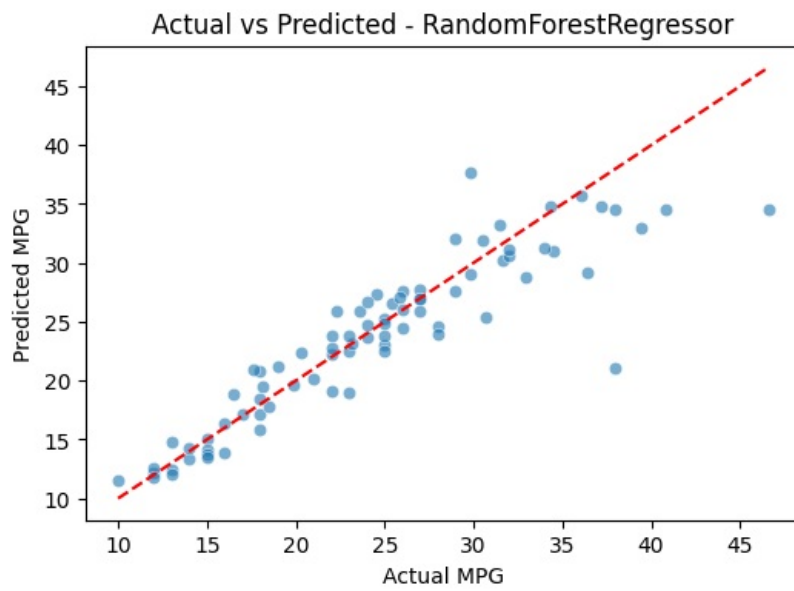


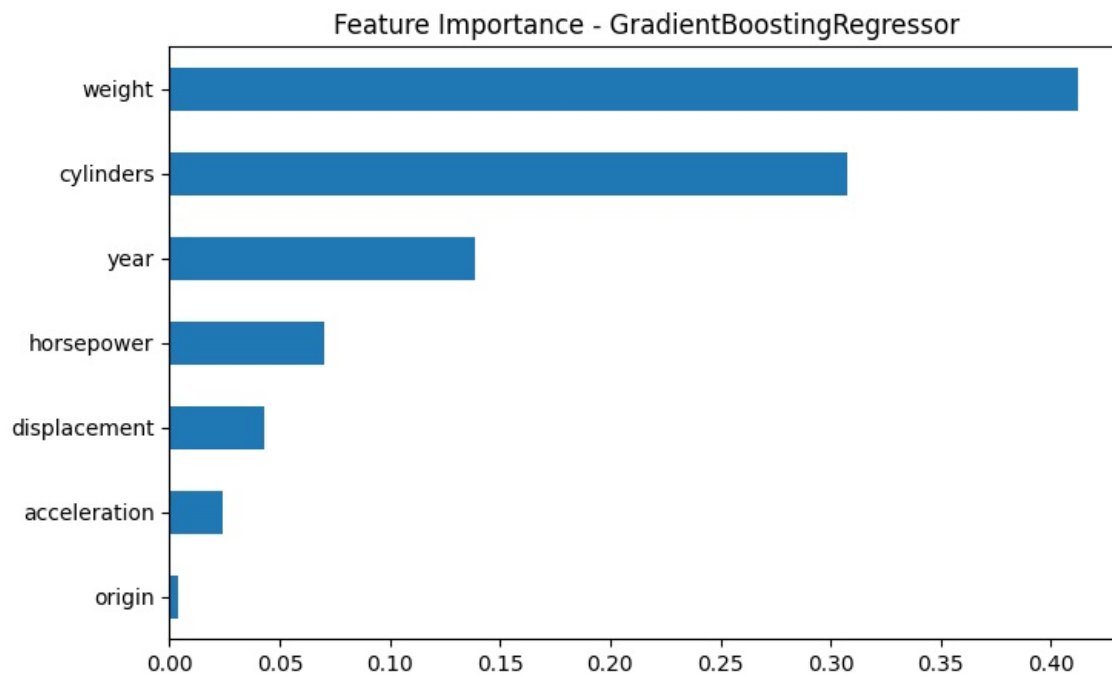
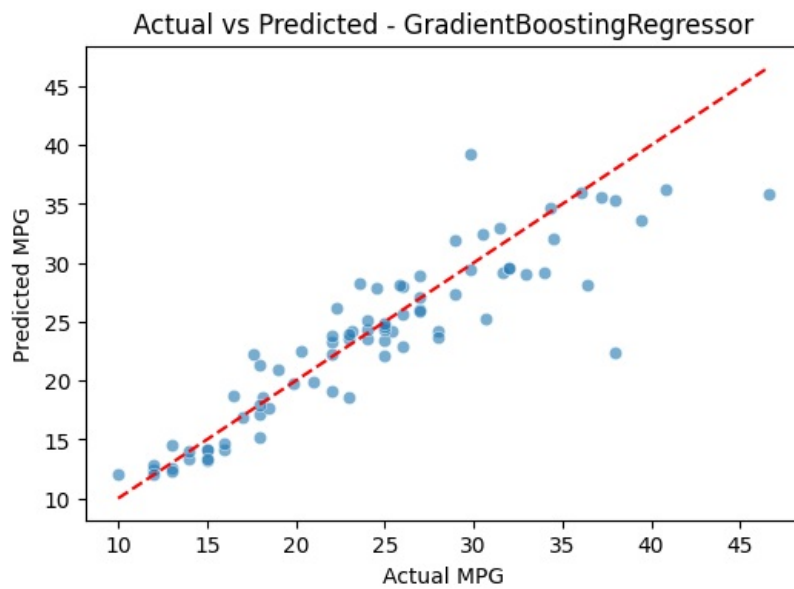
Actual vs Predicted - LinearRegression



Residual Plot - RandomForestRegressor







```
In [246... df_reg_results = pd.DataFrame(reg_results)
print(df_reg_results)
```

	LinearRegression	RandomForestRegressor	GradientBoostingRegressor
Train R^2	0.827605	0.983804	0.977081
Test R^2	0.786332	0.815740	0.812301
Train MSE	10.490042	0.985529	1.394573
Test MSE	13.112707	11.307952	11.518971

Interpretation of Regressors:

We evaluated three different regression models (Linear Regression, Random Forest Regressor, and Gradient Boosting Regressor) on the Auto dataset, with mpg (Miles Per Gallon) as the target variable.

- **R^2 Score:** Measures how well the model explains the variance in the target variable (mpg). A higher R^2 indicates a better fit.
- **MSE (Mean Squared Error):** Measures the average squared difference between actual and predicted values. A lower MSE indicates better performance.

Linear Regression Analysis

- **Simplest model** with low complexity and high interpretability.
- **Train $R^2 = 0.8276$ and Test $R^2 = 0.7863$** , meaning it explains ~83% of training variance but fails to generalize as well on unseen data.
- **Moderate overfitting**, as Train and Test R^2 show a noticeable difference.
- **Train MSE = 10.49 and Test MSE = 13.11**, indicating moderate prediction errors.

Linear Regression provides a strong baseline model with moderate generalization. While it may not capture complex interactions as well

as tree-based models, it remains a reliable choice for interpretability.

Random Forest Regressor Analysis

- **Train $R^2 = 0.9838$** , meaning it captures nearly all variance in training data.
- **Test $R^2 = 0.8157$** , showing better generalization than Linear Regression but some overfitting.
- **Train MSE = 0.9855** and **Test MSE = 11.3080**, showing a significant gap, indicating the model is memorizing training data to some extent.

Random Forest performs well, capturing complex relationships in the data. However, the gap between training and test performance suggests mild overfitting, which could be improved with hyperparameter tuning (e.g., reducing tree depth, increasing regularization).

Gradient Boosting Regressor Analysis

- **Train $R^2 = 0.9771$** , meaning it effectively learns patterns in training data.
- **Test $R^2 = 0.8123$** , showing similar performance to Random Forest but slightly lower generalization.
- **Train MSE = 1.3946** and **Test MSE = 11.5190**, showing the lowest overfitting among the tree-based models.

Gradient Boosting performs almost similarly to Random Forest, achieving strong generalization with slightly higher stability. While its test R^2 is marginally lower than Random Forest, its training performance suggests better regularization and less memorization**.

Random Forest Regressor achieves the best test performance with Test $R^2 = 0.8157$ and Test MSE = 11.3080, making it the most suitable model for predicting mpg in this dataset.

Extra Work (Optional): Classification Models Implementation:

I've implemented a Classification Problem as well for predicting mpg_class (Miles Per Gallon Class) using following classifiers:

This is optional and not required by the assessment.

```
In [247.. class_models = {
    "LogisticRegression": LogisticRegression(),
    "RandomForest": RandomForestClassifier(random_state=10),
    "GradientBoosting": GradientBoostingClassifier(random_state=10),
}

In [248.. # Dictionary to store Train & Test performance
class_results = {}

for name, model in class_models.items():
    if isinstance(model, LogisticRegression):
        model.fit(X_train_class_scaled, y_train_class) # Train on scaled data for Linear Regression

        y_pred_class_train = model.predict(X_train_class_scaled)
        y_pred_class_test = model.predict(X_test_class_scaled)
    else:
        model.fit(X_train_class, y_train_class) # Train on raw (unscaled) data for Tree-based models

        y_pred_class_train = model.predict(X_train_class)
        y_pred_class_test = model.predict(X_test_class)

    # Store Accuracy, F1-score & R² Score for Train & Test
    acc_train_class = accuracy_score(y_train_class, y_pred_class_train)
    acc_test_class = accuracy_score(y_test_class, y_pred_class_test)
    f1_train_class = f1_score(y_train_class, y_pred_class_train)
    f1_test_class = f1_score(y_test_class, y_pred_class_test)

    class_results[name] = {
        "Train Accuracy": acc_train_class, "Test Accuracy": acc_test_class,
        "Train F1": f1_train_class, "Test F1": f1_test_class
    }

    # Print Model Performance
    print(f"\n{name} Performance:")
    print(f"Train Accuracy: {acc_train_class:.2f} | Test Accuracy: {acc_test_class:.2f}")
    print(f"Train F1 Score: {f1_train_class:.2f} | Test F1 Score: {f1_test_class:.2f}")
    print(classification_report(y_test_class, y_pred_class_test))

    # Plot Confusion Matrix for Train & Test
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
```

```

# Train Confusion Matrix
sns.heatmap(confusion_matrix(y_train_class, y_pred_class_train), annot=True, fmt="d",
            cmap="coolwarm", xticklabels=["Low MPG", "High MPG"],
            yticklabels=["Low MPG", "High MPG"], ax=axes[0])

axes[0].set_title(f"Train Confusion Matrix - {name}")
axes[0].set_xlabel("Predicted")
axes[0].set_ylabel("Actual")

# Test Confusion Matrix
sns.heatmap(confusion_matrix(y_test_class, y_pred_class_test), annot=True, fmt="d",
            cmap="coolwarm", xticklabels=["Low MPG", "High MPG"],
            yticklabels=["Low MPG", "High MPG"], ax=axes[1])

axes[1].set_title(f"Test Confusion Matrix - {name}")
axes[1].set_xlabel("Predicted")
axes[1].set_ylabel("Actual")

plt.tight_layout()
plt.show()

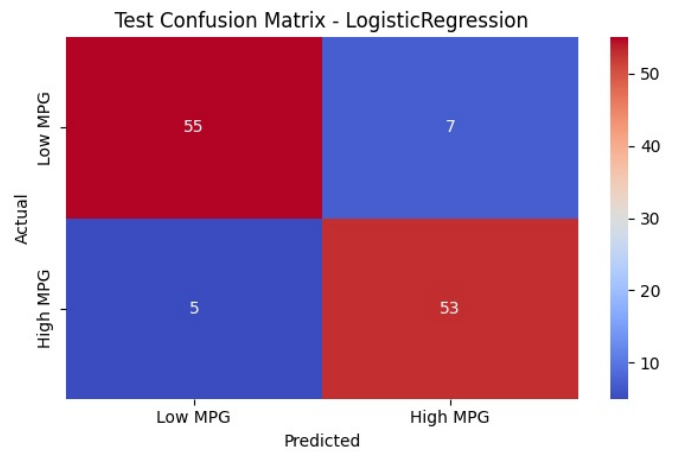
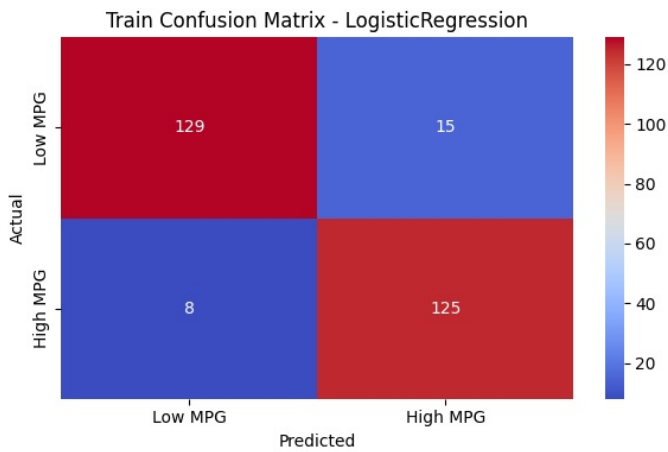
```

LogisticRegression Performance:

Train Accuracy: 0.92 | Test Accuracy: 0.90

Train F1 Score: 0.92 | Test F1 Score: 0.90

	precision	recall	f1-score	support
0	0.92	0.89	0.90	62
1	0.88	0.91	0.90	58
accuracy			0.90	120
macro avg	0.90	0.90	0.90	120
weighted avg	0.90	0.90	0.90	120

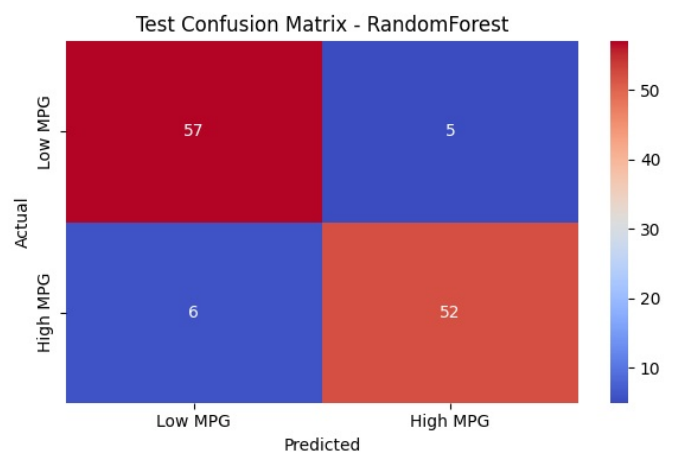
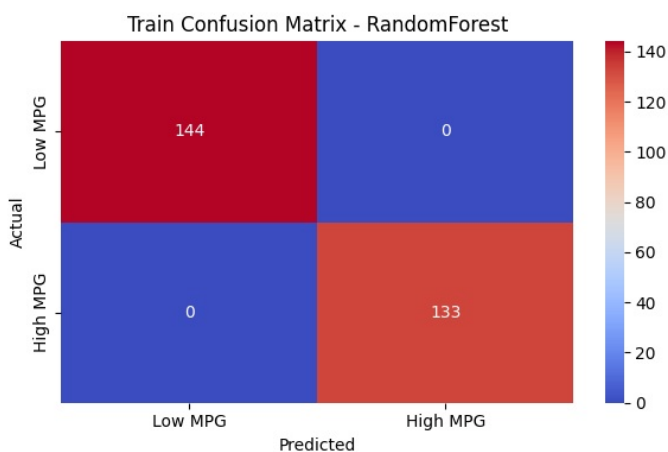


RandomForest Performance:

Train Accuracy: 1.00 | Test Accuracy: 0.91

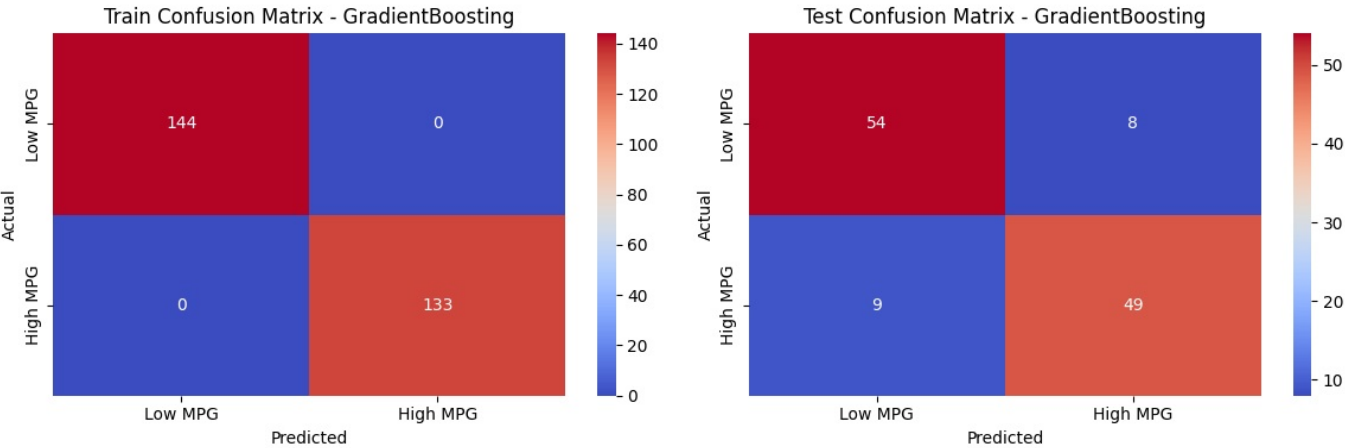
Train F1 Score: 1.00 | Test F1 Score: 0.90

	precision	recall	f1-score	support
0	0.90	0.92	0.91	62
1	0.91	0.90	0.90	58
accuracy			0.91	120
macro avg	0.91	0.91	0.91	120
weighted avg	0.91	0.91	0.91	120



GradientBoosting Performance:
Train Accuracy: 1.00 | Test Accuracy: 0.86
Train F1 Score: 1.00 | Test F1 Score: 0.85

	precision	recall	f1-score	support
0	0.86	0.87	0.86	62
1	0.86	0.84	0.85	58
accuracy			0.86	120
macro avg	0.86	0.86	0.86	120
weighted avg	0.86	0.86	0.86	120



```
In [249]: # Display Results

df_class_results = pd.DataFrame(class_results)

print(df_class_results)
```

	LogisticRegression	RandomForest	GradientBoosting
Train Accuracy	0.916968	1.000000	1.000000
Test Accuracy	0.900000	0.908333	0.858333
Train F1	0.915751	1.000000	1.000000
Test F1	0.898305	0.904348	0.852174

Interpretation of Classifiers:

We evaluated three different classification models (Logistic Regression, Random Forest Classifier, and Gradient Boosting Classifier) on the Auto dataset, with mpg_class (Milage Per Gallon: Low/High) as the target variable.

- **Accuracy:** Measures the proportion of correctly classified instances. Higher accuracy indicates better model performance.
- **F1 Score:** Balances precision and recall to provide a better measure of a model's effectiveness, especially in cases of class imbalance.

Logistic Regression Performance

- Train Accuracy: 0.92, Test Accuracy: 0.90.
- Train F1 Score: 0.92, Test F1 Score: 0.90.
- Precision for class 0 (Low Milage): 0.92, Recall: 0.89, F1 Score: 0.90.
- Precision for class 1 (High Milage): 0.88, Recall: 0.91, F1 Score: 0.90.
- Balanced classification with macro average F1-score of 0.90.
- Maintains similar accuracy in both training and test sets, indicating minimal overfitting.

Logistic Regression provides strong generalization, achieving a high test accuracy of 90%, and effectively balances precision and recall for both classes.

Random Forest Classifier Performance

- Train Accuracy: 1.00, Test Accuracy: 0.91.
- Train F1 Score: 1.00, Test F1 Score: 0.90.
- Precision for class 0 (Low Mileage): 0.90, Recall: 0.92, F1 Score: 0.91.
- Precision for class 1 (High Mileage): 0.91, Recall: 0.90, F1 Score: 0.90.
- Macro average F1-score: 0.91, indicating balanced classification.
- Although the model performed well with 91% accuracy on the test set, the perfect train accuracy (1.00) suggests possible overfitting, as the test accuracy (0.91) is slightly lower. We can further improve generalization and reduce overfitting by applying pruning and adjusting hyperparameters such as max_depth, min_samples_split, and n_estimators.

Random Forest achieves high test accuracy (0.91) and maintains strong F1-scores across both classes.

Gradient Boosting Classifier Performance

- Train Accuracy: 1.00, Test Accuracy: 0.86.
- Train F1 Score: 1.00, Test F1 Score: 0.85.
- Precision for class 0 (Low Mileage): 0.86, Recall: 0.87, F1 Score: 0.86.
- Precision for class 1 (High Mileage): 0.86, Recall: 0.84, F1 Score: 0.85.
- Macro average F1-score: 0.86, showing slightly worse performance than the other models.
- Large gap between train (1.00) and test accuracy (0.86) suggests overfitting, as the model is performing exceptionally well on training data but performed moderately well on unseen data. We can improve generalization by tuning hyperparameters such as `learning_rate`, `n_estimators`, and `max_depth` to control model complexity.

Gradient Boosting captures complex patterns but generalizes poorly compared to other models, leading to a drop in test performance.

Having the highest test accuracy (0.91) but with some overfitting concerns, Random Forest is the best-performing model on this data. However, Logistic Regression remains the most stable and generalizable model, making it the preferred choice if interpretability and consistency are prioritized.

Final Conclusion:

- **Best Regression Model** = Random Forest Regressor Model
- **Best Classification Model** = Logistic Regression Model

Random Forest Regressor achieves the best predictive performance for regression tasks, while Logistic Regression provides the most stable and generalizable results for classification. However, after pruning and hyperparameter tuning, this could change, as Random Forest or Gradient Boosting may improve their generalization and outperform Logistic Regression in classification.
